STUDY SERVER · MANUAL PATH

Installing HCI Playground on the 3090

The proven `setup → start → publish` sequence, one command at a time. Every command is PowerShell, run from the repository root. The box is a one-shot study target — each student sits each topic once — so the gates below refuse rather than warn.

`Windows · PowerShell` `branch: master` `backend 462/0 · e2e 416/0`

BEFORE YOU START

One tool has to come first — **Git**, so you can clone the repo that carries the manifest for everything else. NVIDIA drivers for the 3090 should already be present.

```
winget install -e --id Git.Git --accept-package-agreements --accept-source-agreements
```

THE REST Python, Node, Ollama and cloudflared install in **one command** right after you clone (Step 1), from `deploy\winget-packages.json`. No five-line block.

Then have these ready — you bring them, nothing installs them

- [ ] Your backup of **backend\.participant_secret** and its **16-hex fingerprint**.
- [ ] The real **enrolled_sids.txt** (roster, `SID,section` per line) and **admin_sids.txt** (teacher SIDs).
- [ ] A free **healthchecks.io** ping URL for the heartbeat (Step 8).

THE INSTALL

1 Get the code, then the tools

Clone — the default branch `master` now holds the tested code (backend 462/0, e2e 416/0) — then install Python + Node + Ollama + cloudflared from the manifest in one command.

```
git clone https://github.com/wilson-cheng1110/compgame-fyp.git
cd compgame-fyp
winget import -i deploy\winget-packages.json --accept-package-agreements --accept-source-agreements
```

THEN Everything runs **from this folder** (it holds `backend\`, `deploy\`). **Open a new PowerShell** after the import so PATH picks up what was installed. If an id errors, `winget search ollama` and swap it — `setup.ps1` re-checks anyway.

2 Drop in the three files git will not carry

Copy them into `backend\`. They are gitignored because they hold real people and a real key. Then verify the key is the *right* one:

```
(Get-FileHash backend\.participant_secret -Algorithm SHA256).Hash.Substring(0,16)
```

MUST MATCH The output must equal your backup's fingerprint. A **fresh key silently breaks every pre/post export join** — the publish gate checks the file is *present*, never that it is correct. This step is the only guard.

ALSO `enrolled_sids.txt` = the real roster (`SID,section` per line). `admin_sids.txt` = teacher SIDs, one per line. Without the roster, sign-up is open and students self-report their section.

3 Run setup

The proven one-shot for the machine: it checks the prerequisites, builds the venv and the app, **pulls the Ollama models (~8 GB, once)**, and writes a starter `deploy\.env.local`. Re-runnable — it never overwrites a secret, a database, or an existing `.env.local`, and resumes after a failure.

```
powershell -ExecutionPolicy Bypass -File deploy\setup.ps1
```

IF IT STOPS It refuses when Python / Node / Ollama is missing and prints the link. Install it, **open a NEW PowerShell** so PATH refreshes, and re-run.

4 Set the heartbeat URL

Open `deploy\.env.local` and add your healthchecks.io ping URL. `COOKIE_SECURE=1` is already written; leave `QUESTIONNAIRES_ENABLED` / `TELEMETRY_ENABLED` unset until the HSESC amendment lands.

```
HEARTBEAT_URL=https://hc-ping.com/your-uuid-here
```

5 Start it (loopback)

Brings up the API on 8080 and the app on 3000, both loopback only, and **waits until each actually answers** before returning — "the process started" is not "the service is up".

```
powershell -ExecutionPolicy Bypass -File deploy\start.ps1
```

TO STOP `powershell -ExecutionPolicy Bypass -File deploy\start.ps1 -Stop`

6 Sanity-check before going public

Read the health endpoint *through port 3000* (that proves the rewrite works). On Windows PowerShell use `Invoke-RestMethod` — plain `curl` is an alias there and prints an object, not JSON.

```
Invoke-RestMethod http://127.0.0.1:3000/api/health | ConvertTo-Json -Depth 6
```

CONFIRM `status: ok`, `enrolment` = your cohort size (**not 8000**), `rag_model.ok: true`. Then sign in as a throwaway student, **ask the tutor once and time it** (this box's latency is unmeasured), and run one full topic unit.

7 Publish

Opens the Cloudflare tunnel in front of port 3000. It **refuses on a red gate** and tells you why.

```
powershell -ExecutionPolicy Bypass -File deploy\publish.ps1
```

GATE Refuses unless: web + API answer through :3000, `COOKIE_SECURE≠0`, participant key present, enrolment **≠ 8000**, and the bundle names no hardcoded origin.

TUNNEL Needs `cloudflared`. A named tunnel gives a **stable URL** students can bookmark, but wants a one-time `cloudflared tunnel login`. Note the URL it prints.

8 Make it survive reboots

Run from an **elevated** PowerShell. Registers three Scheduled Tasks: **Boot** (start at startup), **Watchdog** (restart if it stops answering), **Heartbeat** (ping out every 5 min — if the pings stop, healthchecks.io emails you).

```
powershell -ExecutionPolicy Bypass -File deploy\install-services.ps1
```

THEN **Reboot the box once.** Confirm it comes back (health `ok`) and the heartbeat resumes at healthchecks.io — that is the only real proof of "leave it alone".

KNOW THESE GOING IN

- [ ] **Idle timeout 30 min.** A session idle 30 min with zero interaction is logged out; the keep-alive pings on any activity, so active use never expires. Recorded steps survive a logout (progress is server-side); only unsubmitted, in-progress input is lost. Tune with `SESSION_IDLE_MINUTES`.
- [ ] **norman / hicks-law** have thin tutor corpus (2023 decks). Reported separately; not H1 evidence.
- [ ] **Backups:** an hourly online-backup of the sink + accounts runs on its own — confirm it is writing.
- [ ] **A release window is five days wide**, not one morning. An hour of downtime is not a lost topic — do not panic-fix during a window.

The same steps with the full reasoning are in `deploy\3090-bringup.md` (in the repo after Step 1). One command version of Steps 3–5, once the Step 2 files are in place: `deploy\bootstrap.ps1 -SecretFingerprint <16hex>`.